

웹프로그래밍 응용

고민 상담 BAR

React & Tomcat(Springboot)

2019875058 정재훈

2021863026 류연우

웹프로그래밍 응용
김주현 교수님

목차

1. Tomcat
2. SpringBoot
3. MVC
4. 디렉토리 구조
5. 소스코드 설명

Tomcat ??



**Apache
Server?**



**Apache
Tomcat?**

Spring boot ??



실제 사용 예시

기관/분야	시스템	사용
행정안전부	정부24, 전자결재 등	Spring 기반
교육부	에듀파인, 나이스 등	대부분 Spring
보건복지부	복지로, 건강보험공단 등	Spring Framework + JPA
서울시	열린데이터광장, 시민참여포털 등	Spring MVC / Boot
공공데이터포털	data.go.kr API	백엔드 Spring 기반 API 서버
LX공사, 도로공사 등	전산 시스템	Spring 기반 REST 서버 사용



Project

☒ Gradle - Groovy ☐ Gradle - Kotlin
☐ Maven

Language

☒ Java ☐ Kotlin ☐ Groovy

Spring Boot

☐ 4.0.0 (SNAPSHOT) ☐ 3.5.1 (SNAPSHOT) ☒ 3.5.0 ☐ 3.4.7 (SNAPSHOT)
☐ 3.4.6 ☐ 3.3.13 (SNAPSHOT) ☐ 3.3.12

Project Metadata

Group

Artifact

Name

Description

Package name

Packaging ☒ Jar ☐ War

Java ☐ 24 ☐ 21 ☒ 17

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring Web WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Data JPA SQL

Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

MySQL Driver SQL

MySQL JDBC driver.

Thymeleaf TEMPLATE ENGINES

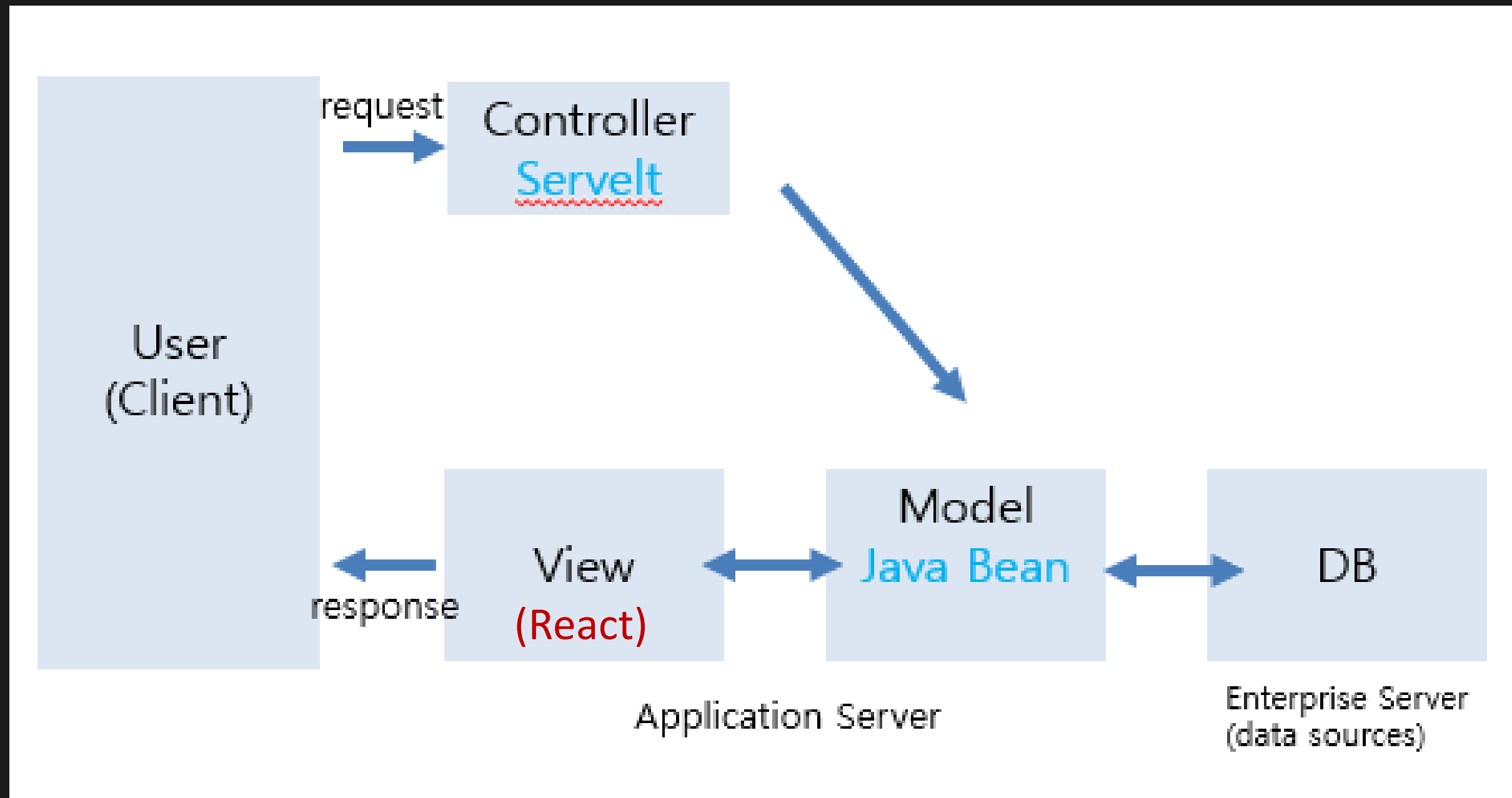
A modern server-side Java template engine for both web and standalone environments. Allows HTML to be correctly displayed in browsers and as static prototypes.

Lombok DEVELOPER TOOLS

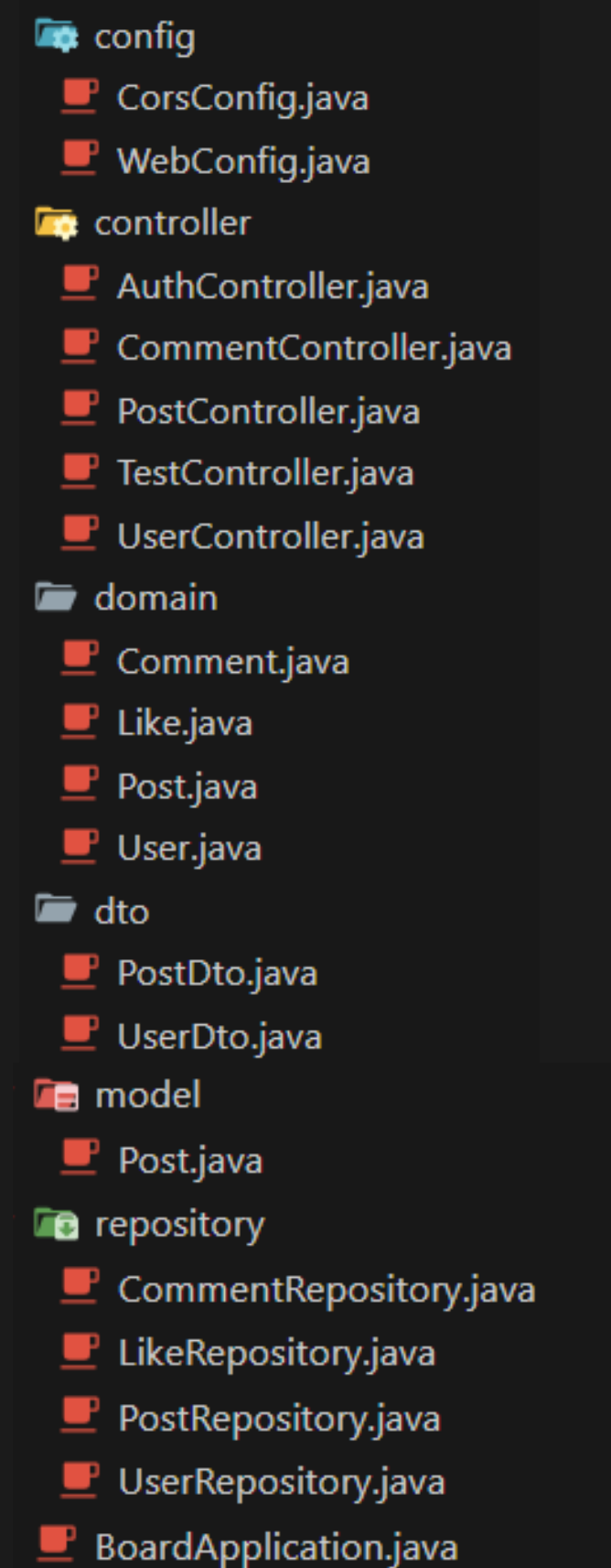
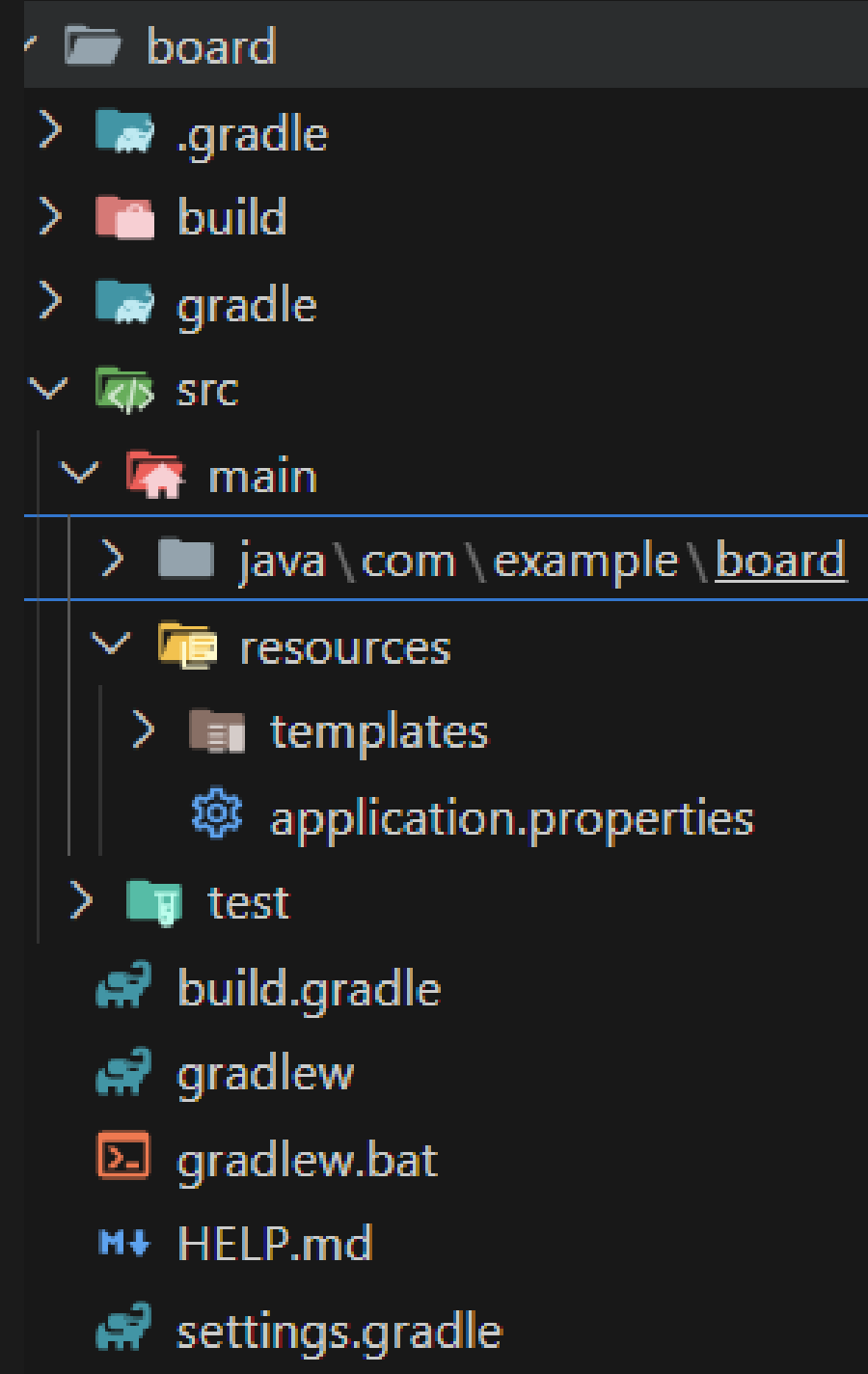
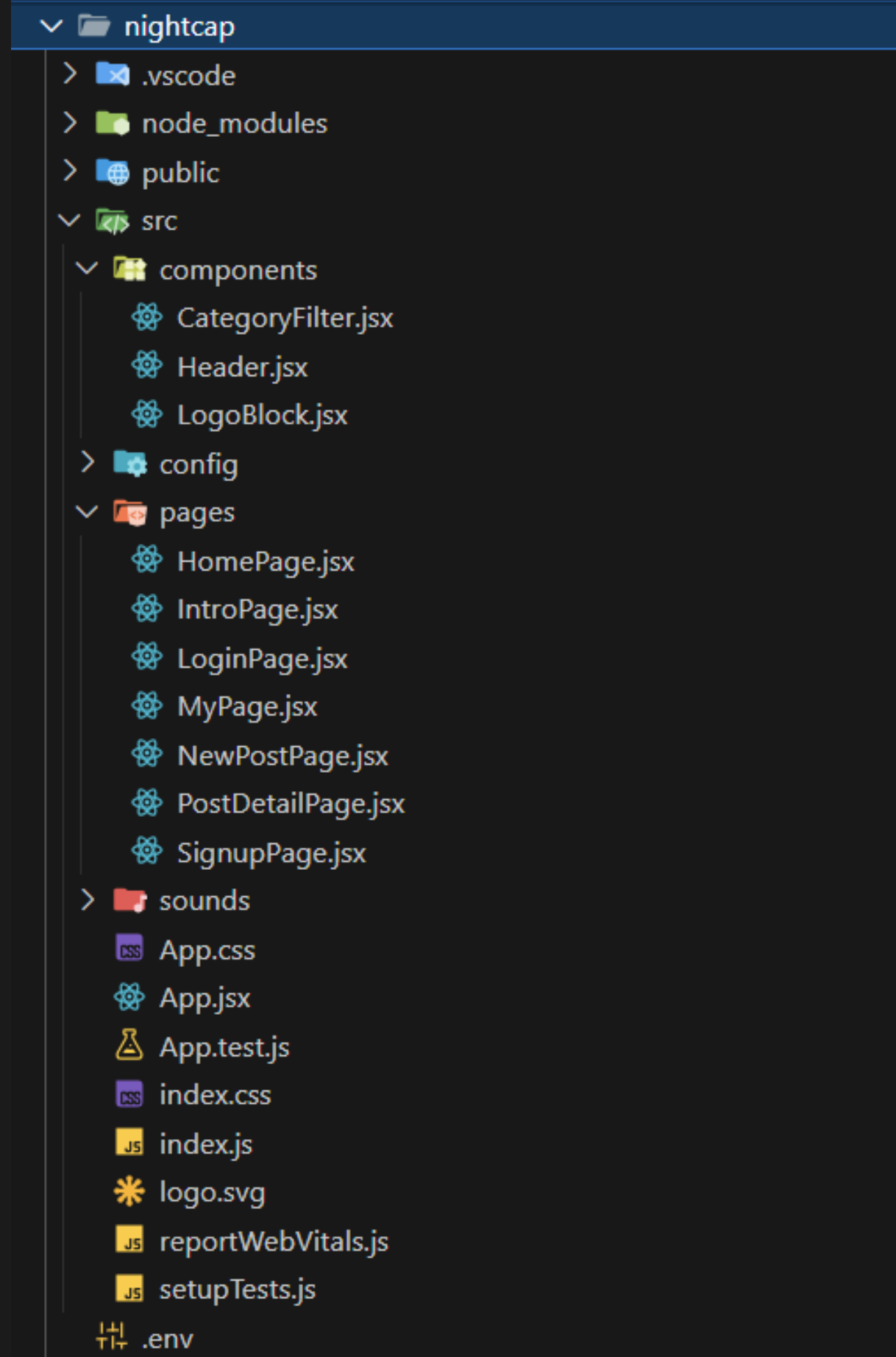
Java annotation library which helps to reduce boilerplate code.

-  Spring Web
-  Spring Data JPA
-  MySQL Driver
-  Thymeleaf
-  Lombok

MVC



디렉토리 구조



- 📁 전체
- 🍷 연애
- 🏠 가정
- 📖 학업
- 🍺 직장
- 🍷 교우
- 🏥 건강
- 🍜 메뉴
- 🥕 당근
- 🍷 TMI

GPT에게 상담 받아보기

GPT에게 고민을 물어보세요...

전송

TMI

👤 마스터

고양이 귀여워 강아지도 귀여워

❤️ 1

💬 0

연애

👤 마스터

그린라이트 인가요?

❤️ 0

💬 0

직장

👤 마스터

상사가 너무 끈대예요

❤️ 0

💬 0

🔍

학업

👤 마스터

힘들어

❤️ 0

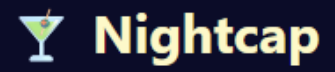
💬 0

건강

👤 마스터

허리가 아파

👉 고민 쓰기



어두운 밤, 익명의 고민과 기발한 조언이 오가는 바

로그인

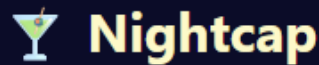
아이디

비밀번호

☐ 로그인 상태 유지

로그인

[회원가입](#)



Nightcap

어두운 밤, 익명의 고민과 기발한 조언이 오가는 바

마스터

TMI

마스터

고양이 귀여워 강아지도 귀여워

수정 삭제

1 1

댓글

마스터 2025. 6. 5.

0

0

인정...

수정 삭제

100자 이내로 댓글 작성

댓글 달기

뒤로가기



마스터님의 마이페이지

내가 남긴 글과 공감, 댓글들을 확인해보세요 😊

로그아웃

📌 내가 쓴 글

고양이 귀여워 강아지도 귀여
고양이 귀여워 강아지도 귀여워

그린라이트 인가요?
그린라이트 인가요?

상사가 너무 끈대예요
상사가 너무 끈대예요

힘들어
힘들어

허리가 아파
허리가 아파

친구가 여자친구가 생겼어요
친구가 여자친구가 생겼어요

💖 공감한 글

고양이 귀여워 강아지도 귀여
고양이 귀여워 강아지도 귀여워

💬 댓글 단 글

고양이 귀여워 강아지도 귀여
고양이 귀여워 강아지도 귀여워

application.properties

spring.datasource.url=jdbc:mysql://localhost:3306/board?serverTimezone=UTC

spring.datasource.username=root

spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect

logging.level.org.hibernate.orm.jdbc.bind=trace

logging.level.org.springframework.orm.jpa=debug

logging.level.org.hibernate=debug

logging.level.org.springframework.web=DEBUG

logging.level.org.hibernate.SQL=DEBUG

logging.level.org.hibernate.type.descriptor.sql=TRACE

Header.jsx

```
return (
  <div className="bg-[#0b0c2a] text-white border-b border-gray-700">
    <div className="relative pt-6 px-4">
      <div className="absolute right-10 top-10 flex items-center gap-2">
        {currentUser ? (
          <>
            <img
              src={
                aliasIcons[getAliasBase(currentUser.alias)] ||
                "/icons/default.png"
              }
              alt="alias"
              className="w-6 h-6"
            />
            <span
              className="text-sm text-blue-300 font-semibold cursor-pointer"
              onClick={() => navigate("/mypage")}
            >
              {currentUser.alias}
            </span>
          </>
        ) : (
          <button onClick={() => navigate("/login")}>
            
          </button>
        )}
      </div>
    </div>

    <div className="flex justify-center">
      <LogoBlock resetCategory={() => setSelectedCategory("전체")} />
    </div>
  </div>
</div>
);
}
```

LogoBlock.jsx

```
import { useNavigate } from "react-router-dom";

export default function LogoBlock({ resetCategory }) {
  const navigate = useNavigate();

  const handleClick = () => {
    if (resetCategory) resetCategory();
    navigate("/home");
  };

  return (
    <div
      onClick={handleClick}
      className="text-center cursor-pointer mb-6"
    >
      <h1 className="text-3xl font-bold text-yellow-100 flex justify-center items-center gap-2">
        🍷 Nightcap
      </h1>
      <p className="text-sm italic text-gray-400 mt-1">
        어두운 밤, 익명의 고민과 기발한 조언이 오가는 바
      </p>
    </div>
  );
}
```

CategoryFilter.jsx

```
import { useLocation } from "react-router-dom";

export default function CategoryFilter({ selectedCategory,
setSelectedCategory }) {
  const location = useLocation();
  const hiddenRoutes = ["/", "/mypage"];

  if (hiddenRoutes.includes(location.pathname)) return null;

  const categoryIcons = {
    전체: "all",
    연애: "love",
    가정: "family",
    학업: "study",
    직장: "work",
    교우: "friends",
    건강: "health",
    메뉴: "menu",
    당근: "carrot",
    TMI: "tmi",
  };

  const allCategories = Object.keys(categoryIcons);
```

CategoryFilter.jsx

```
    return (  
      <div className="flex flex-wrap gap-3 justify-center bg-[#1a1b3a] py-3 px-4">  
        {allCategories.map((cat, i) => (  
          <button  
            key={i}  
            onClick={() => setSelectedCategory(cat)}  
            className={`flex items-center gap-1 px-4 py-1 rounded-full font-bold transition ${  
              selectedCategory === cat  
                ? "bg-blue-400 text-black"  
                : "bg-blue-300 text-black hover:bg-blue-400"  
            }`}   
          >  
            <img  
              src={` /icons/${categoryIcons[cat]}.png`}   
              alt={cat}  
              className="w-5 h-5"   
            />  
            {cat}  
          </button>  
        ))}  
      </div>  
    );  
  }  
}
```

Intro.jsx

```
import { useNavigate } from "react-router-dom";
import LogoBlock from "../components/LogoBlock";

export default function IntroPage() {
  const navigate = useNavigate();

  return (
    <div className="min-h-screen bg-[#0b0c2a] flex flex-col items-
center justify-center text-white text-center px-4">
      <LogoBlock />
      <p className="text-lg mt-4 mb-8 text-gray-300">
        익명의 고민과 따뜻한 조언이 오가는 밤의 바에 오신 것을 환영합니다.
      </p>
      <button
        onClick={() => navigate("/home")}
        className="bg-pink-500 hover:bg-pink-600 px-6 py-3 rounded-full
text-lg font-bold transition"
      >
        🍷 입장하기
      </button>
    </div>
  );
}
```

App.jsx

```
import { Routes, Route, useLocation } from "react-router-dom";
import { useEffect, useState } from "react";
import HomePage from "./pages/HomePage";
import LoginPage from "./pages/LoginPage";
import PostDetailPage from "./pages/PostDetailPage";
import NewPostPage from "./pages/NewPostPage";
import SignupPage from "./pages/SignupPage";
import Header from "./components/Header";
import MyPage from "./pages/MyPage";
import "./App.css";
import CategoryFilter from "./components/CategoryFilter";
import IntroPage from "./pages/IntroPage";

export default function App() {
  const location = useLocation();
  const hideHeaderRoutes = ["/", "/login", "/signup", "/mypage"];
  const shouldHideHeader =
    hideHeaderRoutes.includes(location.pathname);
  const [posts, setPosts] = useState([]);
  const [isLoggedIn, setIsLoggedIn] = useState(false);
  const [authToken, setAuthToken] = useState(null);
  const [userId, setUserId] = useState(null);
  const [selectedCategory, setSelectedCategory] = useState("전체");
  const [currentUser, setCurrentUser] = useState(null);
```

App.jsx

```
useEffect(() => {
  fetch("http://localhost:8080/posts")
    .then((res) => res.json())
    .then((data) => {
      console.log("서버에서 받은 posts:", data);
      setPosts(data);
    });
}, []);

useEffect(() => {
  const savedUser = localStorage.getItem("user");
  const loginTime = localStorage.getItem("loginTime");

  const SESSION_DURATION = 1000 * 60 * 60; // 1시간

  if (savedUser && loginTime) {
    const now = Date.now();
    const elapsed = now - parseInt(loginTime, 10);

    if (elapsed < SESSION_DURATION) {
      const parsed = JSON.parse(savedUser);
      setIsLoggedIn(true);
      setCurrentUser(parsed);
      setUserId(parsed.id);
    } else {
      // ⌚ 세션 만료
      localStorage.removeItem("user");
      localStorage.removeItem("loginTime");
      setIsLoggedIn(false);
      setCurrentUser(null);
      setUserId(null);
      alert("세션이 만료되어 자동 로그아웃 되었습니다.");
    }
  }
}, []);
```

```
useEffect(() => {
  const saved =
    JSON.parse(localStorage.getItem("user")) ||
    JSON.parse(sessionStorage.getItem("user"));

  if (saved) {
    setIsLoggedIn(true);
    setCurrentUser(saved);
    setUserId(saved.id);
  }
}, []);
```

App.jsx

```
return (
  <div className="App">
    {!shouldHideHeader && (
      <Header
        currentUser={currentUser}
        selectedCategory={selectedCategory}
        setSelectedCategory={setSelectedCategory}
      />
    )}
    {!["/new", "/signup", "/login"].includes(location.pathname) &&
    !location.pathname.startsWith("/posts") && (
      <CategoryFilter
        selectedCategory={selectedCategory}
        setSelectedCategory={setSelectedCategory}
      />
    )}
    <Routes>
      <Route path="/" element={<IntroPage />} />
      <Route
        path="/home"
        element={
          <HomePage
            posts={posts}
            setPosts={setPosts}
            isLoggedIn={isLoggedIn}
            currentUser={currentUser}
            selectedCategory={selectedCategory}
          />
        } />
      <Route
        path="/login"
        element={
          <LoginPage
            setIsLoggedIn={setIsLoggedIn}
            setUserId={setUserId}
            setCurrentUser={setCurrentUser}
          />
        } />
    </Routes>
  </div>
);
```

```
<Route path="/signup" element={<SignupPage />} />
<Route
  path="/posts/:postId"
  element={
    <PostDetailPage
      posts={posts}
      setPosts={setPosts}
      isLoggedIn={isLoggedIn}
      authToken={authToken}
      userId={userId}
      currentUser={currentUser}
    />
  } />
<Route
  path="/new"
  element={
    <NewPostPage
      setPosts={setPosts}
      isLoggedIn={isLoggedIn}
      userId={userId}
      currentUser={currentUser}
    />
  } />
<Route
  path="/mypage"
  element={<MyPage currentUser={currentUser}
  isLoggedIn={isLoggedIn} />}
/></Routes>
</div>
);
```

Header.jsx

```
import { useNavigate } from "react-router-dom";
import LogoBlock from "../LogoBlock";

export default function Header({
  setSelectedCategory,
  currentUser,
}) {
  const navigate = useNavigate();

  const aliasIcons = {
    밤손님: "/icons/night.png",
    마스터: "/icons/wizard.png",
    요정: "/icons/fairy.png",
    바텐더: "/icons/bartender.png",
    해결사: "/icons/detective.png",
  };

  const getAliasBase = (alias) => alias?.match(/^[\d^]+/)[0];
```

Homepage.jsx

```
import { useState, useEffect } from "react";
import { useNavigate, useLocation } from "react-router-dom";

export default function HomePage({ posts, setPosts, isLoggedIn,
  currentUser, selectedCategory }) {
  const navigate = useNavigate();
  const location = useLocation();
  const [chatLog, setChatLog] = useState([]);
  const [input, setInput] = useState("");
  const [loading, setLoading] = useState(false);
  const [likedPosts, setLikedPosts] = useState({});

  const categoryIcons = {
    연애: "love",
    가정: "family",
    학업: "study",
    직장: "work",
    교우: "friends",
    건강: "health",
    메뉴: "menu",
    당근: "carrot",
    TMI: "tmi",
  };

  useEffect(() => {
    const fetchPosts = async () => {
      try {
        const res = await fetch("http://localhost:8080/posts");
        if (!res.ok) return;
        const data = await res.json();
        setPosts(Array.isArray(data) ? data : []);
      } catch {}
    };

    fetchPosts();
  }, [location.pathname]);
```

```
const handleGPTSend = async () => {
  if (!input.trim() || loading) return;
  setLoading(true);

  const userMessage = { role: "user", content: input };
  const newChatLog = [...chatLog, userMessage];

  try {
    const res = await
    fetch("https://api.openai.com/v1/chat/completions", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer
        ${process.env.REACT_APP_OPENAI_API_KEY}`,
      },
      body: JSON.stringify({
        model: "gpt-3.5-turbo",
        messages: newChatLog,
      }),
    });

    const data = await res.json();
    const botMessage = data.choices?.[0]?.message;
    setChatLog([...newChatLog, botMessage]);
  } catch {}

  setInput("");
  setLoading(false);
};
```

Homepage.jsx

```
const handleTogglePostLike = async (postId) => {
  if (!isLoggedIn || !currentUser?.id) return;

  try {
    const res = await fetch(
      `http://localhost:8080/posts/${postId}/like?userId=${currentUser.id}`,
      {
        method: "PUT",
        credentials: "include",
      }
    );
    const result = await res.text();

    setPosts((prev) =>
      prev.map((p) =>
        p.id === postId
          ? {
              ...p,
              likes: result === "liked" ? (p.likes || 0) + 1 : Math.max((p.likes || 1) - 1, 0),
            }
          : p
        )
    );

    setLikedPosts((prev) => ({
      ...prev,
      [postId]: result === "liked",
    }));
  } catch {}
};
```

```
const filteredPosts = Array.isArray(posts)
  ? selectedCategory === "전체"
    ? posts
    : posts.filter((post) => post.category === selectedCategory)
  : [];
```

Homepage.jsx

```
return (
  <div className="bg-[#0b0c2a] min-h-screen text-white">
    <button
      onClick={() => {
        if (!isLoggedIn) {
          window.location.href = "/login";
          return;
        }
        navigate("/new");
      }}
      className="fixed bottom-6 right-6 bg-pink-500 hover:bg-pink-600 text-white font-bold py-2 px-4 rounded-
full shadow-lg transition"
    >
      🙋 고민 쓰기
    </button>

    <div className="bg-[#1a1b3a] p-4 rounded-md mb-6">
      <h3 className="text-lg font-bold mb-3">GPT에게 상담 받아보기</h3>
      <div className="h-64 overflow-y-auto bg-[#0b0c2a] p-3 rounded mb-3 space-y-2">
        {chatLog.map((msg, i) => (
          <div key={i} className={`flex ${msg.role === "user" ? "justify-end" : "justify-start"}`}>
            <div
              className={`max-w-[80%] p-3 rounded-lg text-sm ${msg.role === "user"
                ? "bg-pink-500 text-white rounded-br-none"
                : "bg-gray-700 text-white rounded-bl-none"
              }`}
            >
              <strong className="block mb-1">{msg.role === "user" ? "L" : "GPT"}</strong>
              <p className="whitespace-pre-wrap">{msg.content}</p>
            </div>
          </div>
        ))}
      </div>

      <div className="flex gap-2">
        <input
          type="text"
          value={input}
          onChange={(e) => setInput(e.target.value)}
          className="flex-grow p-2 rounded bg-[#2c2d4f] text-white"
          placeholder="GPT에게 고민을 물어보세요..."
        />
        <button
          onClick={handleGPTSend}
          className="bg-pink-500 hover:bg-pink-600 px-4 py-2 rounded font-bold"
        >
          전송
        </button>
      </div>
    </div>
  </div>
)
```

Homepage.jsx

```
<div className="px-6 pt-6 space-y-6">
  {filteredPosts.length === 0 ? (
    <div className="text-center py-10 text-gray-400">
      아직 게시물이 없습니다. 첫 게시물을 작성해보세요!
    </div>
  ) : (
    filteredPosts.map((post) => (
      <div
        key={post.id}
        className="bg-[#1a1b3a] p-4 rounded-xl shadow-md cursor-pointer"
        onClick={() => navigate(`/posts/${post.id}`)}
      >
        <div className="flex justify-between items-center text-sm text-gray-400">
          <div className="flex items-center gap-1 bg-blue-300 px-2 py-1 rounded-full text-sm text-black font-medium">
            <img
              src={` /icons/${categoryIcons[post.category] ?? "default"}.png`}
              className="w-4 h-4"
              alt="category"
            />
            <span>{post.category}</span>
          </div>
          <span>{post.category}</span>
        </div>
        <div className="flex items-center gap-1">
          <img
            src={post.profileIcon || "/icons/default.png"}
            className="w-5 h-5"
            alt="icon"
          />
```

Homepage.jsx

```
<span>{post.author || post.authorAlias || "익명"}</span>
  </div>
</div>
<div className="text-base my-2">{post.content}</div>
<div className="flex gap-4 text-sm items-center">
  <button
    className={
      likedPosts[post.id]
        ? "text-pink-400"
        : "text-blue-300 hover:text-blue-400"
    }
    onClick={(e) => {
      e.stopPropagation();
      handleTogglePostLike(post.id);
    }}
  >
    📖 {post.likes ?? 0}
  </button>
  <span>
    💬 {Array.isArray(post.comments) ?
post.comments.length : 0}
  </span>
</div>
</div>
))
)}
</div>
</div>
);
}
```

LogoBlock.jsx

```
import { useNavigate } from "react-router-dom";

export default function LogoBlock({ resetCategory }) {
  const navigate = useNavigate();

  const handleClick = () => {
    if (resetCategory) resetCategory();
    navigate("/home");
  };

  return (
    <div
      onClick={handleClick}
      className="text-center cursor-pointer mb-6"
    >
      <h1 className="text-3xl font-bold text-yellow-100 flex justify-
center items-center gap-2">
        🍷 Nightcap
      </h1>
      <p className="text-sm italic text-gray-400 mt-1">
        어두운 밤, 익명의 고민과 기발한 조언이 오가는 바
      </p>
    </div>
  );
}
```

LoginPage.jsx

```
import { useState, useEffect, useRef } from "react";
import LogoBlock from "../components/LogoBlock";
import { useNavigate } from "react-router-dom";

export default function LoginPage({
  setIsLoggedIn,
  setUserId,
  setCurrentUser,
}) {
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");
  const [errorMsg, setErrorMsg] = useState("");
  const [loginSuccessMsg, setLoginSuccessMsg] = useState("");
  const [keepLoggedIn, setKeepLoggedIn] = useState(false);
  const navigate = useNavigate();
  const usernameRef = useRef(null);

  useEffect(() => {
    usernameRef.current?.focus();
  }, []);
```

LoginPage.jsx

```
const handleLogin = async (e) => {
  e.preventDefault();
  setErrorMsg("");

  try {
    const res = await fetch("http://localhost:8080/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ username, password }),
    });

    if (!res.ok) {
      setErrorMsg("아이디 또는 비밀번호가 틀렸습니다.");
      return;
    }

    const user = await res.json();
    setCurrentUser(user);
    setIsLoggedIn(true);
    setUserId(user.id);
    setLoginSuccessMsg("로그인 완료!");

    const storage = keepLoggedIn ? localStorage : sessionStorage;
    storage.setItem("user", JSON.stringify(user));
    storage.setItem("loginTime", Date.now());

    setTimeout(() => {
      setLoginSuccessMsg("");
      navigate("/home");
    }, 1000);
  } catch (error) {
    setErrorMsg("서버와 통신 중 오류가 발생했습니다.");
  }
};
```

LoginPage.jsx

```
return (
  <div className="min-h-screen bg-[#0b0c2a] text-white flex flex-col items-center justify-center px-4">
    {loginSuccessMsg && (
      <div className="fixed top-4 right-4 bg-green-500 text-white px-4 py-2 rounded shadow-lg z-50
transition">
        {loginSuccessMsg}
      </div>
    )}
    <LogoBlock />
    <div className="bg-[#1a1b3a] p-6 rounded-xl w-full max-w-sm space-y-4 shadow-lg">
      <h2 className="text-xl font-semibold text-center">로그인</h2>
      <form className="space-y-4" onSubmit={handleLogin}>
        <div>
          <label className="block mb-1 text-sm font-medium">아이디</label>
          <input
            ref={usernameRef}
            type="text"
            className="w-full p-2 rounded bg-white text-black"
            required
            value={username}
            onChange={(e) => {
              const onlyEnglish = e.target.value.replace(/^[^a-zA-Z0-9]/g, "");
              setUsername(onlyEnglish);
            }}
          />
        </div>
      </form>
    </div>
  </div>
)
```

LoginPage.jsx

```
<div>
  <label className="block mb-1 text-sm font-medium">비밀번호</label>
  <input
    type="password"
    className="w-full p-2 rounded bg-white text-black"
    required
    value={password}
    onChange={(e) => {
      const noKorean = e.target.value.replace(/[\ㄱ-ㅎㅏ-ㅣ가-힣]/g, "");
      setPassword(noKorean);
    }}
  />
</div>

<div className="flex items-center gap-2 text-sm text-white">
  <input
    type="checkbox"
    id="keepLoggedIn"
    checked={keepLoggedIn}
    onChange={() => setKeepLoggedIn(!keepLoggedIn)}
  />
  <label htmlFor="keepLoggedIn">로그인 상태 유지</label>
</div>
```

LoginPage.jsx

```
<button
    type="submit"
    className="w-full bg-blue-500 hover:bg-blue-400 text-
white font-bold py-2 rounded"
  >
    로그인
  </button>
  {errorMsg && <p className="text-red-400 text-
center">{errorMsg}</p>}}

</form>
<button
  className="mt-4 w-full text-sm text-blue-300
hover:underline"
  onClick={() => navigate("/signup")}
  >
    회원가입
  </button>
</div>
</div>
);
}
```

MyPage.jsx

```
import { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";

export default function MyPage({ currentUser, isLoggedIn }) {
  const [myPosts, setMyPosts] = useState([]);
  const [likedPosts, setLikedPosts] = useState([]);
  const [commentedPosts, setCommentedPosts] = useState([]);

  const navigate = useNavigate();

  const getAliasBase = (alias) => {
    return alias?.match(/^([^\d]+)?\.[0] || "기본");
  };

  const aliasIcons = {
    밤손님: "/icons/night.png",
    마스터: "/icons/wizard.png",
    요정: "/icons/fairy.png",
    바텐더: "/icons/bartender.png",
    해결사: "/icons/detective.png",
  };
};
```

MyPage.jsx

```
useEffect(() => {
  if (!isLoggedIn || !currentUser?.id) return;

  fetch(`http://localhost:8080/users/${currentUser.id}/posts`)
    .then((res) => res.json())
    .then((data) => setMyPosts(Array.isArray(data) ? data : []));

  fetch(`http://localhost:8080/users/${currentUser.id}/liked-posts`)
    .then((res) => res.json())
    .then((data) => setLikedPosts(Array.isArray(data) ? data : []));

  fetch(`http://localhost:8080/users/${currentUser.id}/commented-posts`)
    .then((res) => res.json())
    .then((data) => setCommentedPosts(Array.isArray(data) ? data : []));
}, [currentUser, isLoggedIn]);
```

MyPage.jsx

```
const Section = ({ title, posts, emptyMessage }) => (  
  <section className="space-y-4">  
    <h2 className="text-xl font-bold text-pink-300">{title}</h2>  
    {posts.length === 0 ? (  
      <p className="text-gray-400">{emptyMessage}</p>  
    ) : (  
      <div className="grid gap-4 sm:grid-cols-2 md:grid-cols-3">  
        {posts.map((post) => (  
          <div  
            key={post.id}  
            onClick={() => navigate(`/posts/${post.id}`)}  
            className="cursor-pointer bg-[#1a1b3a] p-4 rounded-xl shadow hover:shadow-lg hover:bg-[#25274a]  
transition"  
          >  
            <div className="text-sm text-gray-400 mb-1">{post.title}</div>  
            <div className="text-white">{post.content}</div>  
          </div>  
        )}}  
      </div>  
    )}  
  </section>  
);
```

MyPage.jsx

```
return (
  <div className="min-h-screen bg-[#0b0c2a] text-white px-6 py-10">
    <div className="w-2/3 mx-auto space-y-12">
      <div className="bg-[#1a1b3a] p-6 rounded-xl shadow flex
justify-between items-center">
        <div>
          <h1 className="text-2xl font-bold text-white flex items-
center gap-2">
            <img
              src={
                aliasIcons[getAliasBase(currentUser?.alias)] ||
                "/icons/default.png"
              }
              alt="icon"
              className="w-6 h-6"
            />
            {currentUser?.alias || "사용자"}님의 마이페이지
          </h1>
          <p className="text-sm text-gray-400 mt-1">
            내가 남긴 글과 공감, 댓글들을 확인해보세요 😊
          </p>
        </div>

        <button
          onClick={() => {
            localStorage.removeItem("user");
            window.location.href = "/login";
          }}
          className="bg-red-500 hover:bg-red-600 text-white px-4 py-2
rounded font-semibold text-sm transition"
        >
          로그아웃
        </button>
      </div>
    </div>
  </div>
);
```

MyPage.jsx

```
<Section
  title="📌 내가 쓴 글"
  posts={myPosts}
  emptyMessage="작성한 글이 없습니다."
/>

<div className="border-t border-gray-700 my-8 w-2/3 mx-auto" />

<Section
  title="💖 공감한 글"
  posts={likedPosts}
  emptyMessage="공감한 글이 없습니다."
/>

<div className="border-t border-gray-700 my-8 w-2/3 mx-auto" />

<Section
  title="💬 댓글 단 글"
  posts={commentedPosts}
  emptyMessage="댓글 단 글이 없습니다."
/>
</div>
</div>
);
}
```

NewPostPage.jsx

```
import { useNavigate, useLocation } from "react-router-dom";

export default function NewPostPage({ currentUser, setPosts }) {
  const navigate = useNavigate();
  const location = useLocation();
  const editingPost = location.state?.post || null;

  const categories = ["연애", "가정", "학업", "직장", "교우", "건강", "메뉴", "당근", "TMI"];
  const [category, setCategory] = useState(editingPost?.category || "연애");
  const [content, setContent] = useState(editingPost?.content || "");

  useEffect(() => {
    if (!currentUser?.id) {
      alert("로그인이 필요합니다.");
      navigate("/login");
    }
  }, [currentUser, navigate]);

  const getAliasIcon = (alias = "") => {
    const base = alias.match(/^[^\d]+/)?.[0] || "";
    const icons = {
      밤손님: "/icons/night.png",
      마스터: "/icons/wizard.png",
      요정: "/icons/fairy.png",
      바텐더: "/icons/bartender.png",
      해결사: "/icons/detective.png",
    };
    return icons[base] || "/icons/default.png";
  };
};
```

NewPostPage.jsx

```
const handleSubmit = async (e) => {
  e.preventDefault();

  if (!content.trim()) {
    alert("고민 내용을 입력해주세요.");
    return;
  }

  const postData = {
    category,
    content,
    title: content.slice(0, 15),
    authorAlias: currentUser?.alias || "익명",
    userId: currentUser?.id || null,
    profileIcon: getAliasIcon(currentUser?.alias),
  };
};
```

NewPostPage.jsx

```
try {
  if (editingPost) {
    const res = await fetch(`http://localhost:8080/posts/${editingPost.id}`, {
      method: "PATCH",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        content,
        title: content.slice(0, 15),
        category,
      }),
    });

    if (!res.ok) throw new Error("수정 실패");
    alert("수정이 완료되었습니다.");
  } else {
    const res = await fetch("http://localhost:8080/posts", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(postData),
    });

    if (!res.ok) throw new Error("등록 실패");
    const newPost = await res.json();

    setPosts((prev) => [...prev, newPost]);

    alert("등록이 완료되었습니다.");
  }

  navigate("/home");
} catch (err) {
  console.error("오류:", err);
  alert("서버 오류가 발생했습니다.");
}
};
```

NewPostPage.jsx

```
return (  
  <div className="bg-[#0b0c2a] min-h-screen text-white px-6 py-10">  
    <div className="flex justify-between items-center mb-6">  
      <h2 className="text-2xl font-bold">  
        {editingPost ? "게시글 수정" : "새 고민 작성"}  
      </h2>  
      <button  
        type="button"  
        onClick={() => navigate("/home")}  
        className="bg-gray-600 hover:bg-gray-700 px-4 py-2 rounded-md font-bold"  
      >  
        뒤로가기  
      </button>  
    </div>  
  
    <form onSubmit={handleSubmit} className="space-y-4 mb-10">  
      <div>  
        <label className="block mb-1">카테고리 선택</label>  
        <select  
          value={category}  
          onChange={(e) => setCategory(e.target.value)}  
          className="w-full p-3 rounded bg-[#1a1b3a] text-white"  
        >  
          {categories.map((cat) => (  
            <option key={cat} value={cat}>  
              {cat}  
            </option>  
          ))}  
        </select>  
      </div>  
    </form>  
  </div>  
)
```

NewPostPage.jsx

```
<textarea
  value={content}
  onChange={(e) => setContent(e.target.value)}
  rows={5}
  placeholder="고민 내용을 입력하세요..."
  className="w-full bg-[#1a1b3a] p-3 rounded-md text-white placeholder-gray-400"
/>

<button
  type="submit"
  className="bg-pink-500 hover:bg-pink-600 px-4 py-2 rounded-md font-bold"
>
  {editingPost ? "수정하기" : "등록하기"}
</button>
</form>
</div>
);
}
```

PostDetailPage.jsx

```
import { useState, useEffect } from "react";
import { useParams, useNavigate } from "react-router-dom";

export default function PostDetailPage({ posts, isLoggedIn, currentUser }) {
  const { postId } = useParams();
  const navigate = useNavigate();
  const postIdNum = parseInt(postId, 10);
  const post = posts.find((p) => p.id === postIdNum);
  const [comments, setComments] = useState([]);
  const [commentReactions, setCommentReactions] = useState({});

  useEffect(() => {
    if (!post && posts.length > 0) {
      alert("해당 포스트를 찾을 수 없습니다.");
      navigate("/");
    }
  }, [post, navigate, posts]);
}
```

PostDetailPage.jsx

```
const handleCommentReaction = (commentId, type) => {
  const currentReaction = commentReactions[commentId];
  setCommentReactions((prev) => {
    if (currentReaction === type) {
      const updated = { ...prev };
      delete updated[commentId];
      return updated;
    } else {
      return { ...prev, [commentId]: type };
    }
  });

  setComments((prev) =>
    prev.map((c) => {
      if (c.id !== commentId) return c;
      let updatedLikes = c.likes || 0;
      let updatedDislikes = c.dislikes || 0;

      if (currentReaction === type) {
        if (type === "like") updatedLikes--;
        else updatedDislikes--;
      } else if (currentReaction === "like" && type === "dislike") {
        updatedLikes--;
        updatedDislikes++;
      } else if (currentReaction === "dislike" && type === "like") {
        updatedDislikes--;
        updatedLikes++;
      } else if (!currentReaction) {
        if (type === "like") updatedLikes++;
        else updatedDislikes++;
      }

      return { ...c, likes: updatedLikes, dislikes: updatedDislikes };
    })
  );
};
```

PostDetailPage.jsx

```
const handleAddComment = async (e) => {
  e.preventDefault();
  const commentText = e.target.comment.value.trim();
  if (!commentText || !currentUser) return;

  const newComment = {
    postId: postIdNum,
    userId: currentUser.id,
    authorAlias: currentUser.alias,
    profileIcon: getAliasIcon(currentUser.alias),
    content: commentText,
  };

  try {
    const res = await
    fetch("http://localhost:8080/comments", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(newComment),
    });

    if (res.ok) {
      const saved = await res.json();
      setComments((prev) => [...prev, saved]);
      e.target.reset();
    } else {
      alert("댓글 등록 실패");
    }
  } catch (err) {
    console.error("댓글 저장 중 오류:", err);
  }
};
```

```
const handleEditComment = async (commentId, currentContent) => {
  const newContent = prompt("댓글을 수정하세요", currentContent);
  if (!newContent || newContent.trim() === "") return;

  try {
    const res = await
    fetch(`http://localhost:8080/comments/${commentId}`, {
      method: "PATCH",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ content: newContent }),
    });

    if (!res.ok) throw new Error("수정 실패");

    setComments((prev) =>
      prev.map((c) =>
        c.id === commentId ? { ...c, content: newContent } : c
      )
    );
  } catch (err) {
    console.error("댓글 수정 오류:", err);
    alert("댓글 수정에 실패했습니다.");
  }
};

const handleDeleteComment = async (commentId) => {
  if (!window.confirm("댓글을 삭제하시겠습니까?")) return;

  try {
    const res = await
    fetch(`http://localhost:8080/comments/${commentId}`, {
      method: "DELETE",
    });

    if (!res.ok) throw new Error("삭제 실패");

    setComments((prev) => prev.filter((c) => c.id !== commentId));
  } catch (err) {
    console.error("댓글 삭제 오류:", err);
    alert("댓글 삭제에 실패했습니다.");
  }
};
```

PostDetailPage.jsx

```
return (  
  <div className="bg-[#0b0c2a] min-h-screen text-white px-6 py-4">  
    <div className="bg-[#1a1b3a] rounded-xl p-6 shadow-md">  
      <div className="flex justify-between items-center text-sm text-gray-400 mb-1">  
        <span>{post.category}</span>  
        <div className="flex items-center gap-2 text-sm font-semibold text-blue-300">  
          <img  
            src={post.profileIcon || "/icons/default.png"}  
            className="w-5 h-5"  
            alt="icon"  
          />  
          <span>{post.author || post.authorAlias}</span>  
        </div>  
      </div>  
  
      <div className="text-base leading-relaxed mb-3">{post.content}</div>  
  
      {isPostAuthor() && (  
        <div className="flex justify-end gap-3 mb-2 text-sm text-gray-400">  
          <button  
            onClick={() => navigate("/new", { state: { post } })}  
            className="hover:text-yellow-300"  
          />  
             수정  
          </button>  
          <button onClick={handleDeletePost} className="hover:text-red-300">  
             삭제  
          </button>  
        </div>  
      )}  
    </div>  
  </div>  
)}
```

PostDetailPage.jsx

```
<div className="flex gap-4 text-sm items-center mb-4">
  <span>💖 {post.likes || 0}</span>
  <span>💬 {comments.length}</span>
</div>

<hr className="border-gray-700 mb-4" />
<h3 className="mb-3 text-lg font-semibold">댓글</h3>

{comments.length === 0 && (
  <p className="text-gray-400">아직 댓글이 없습니다.</p>
)}
```

PostDetailPage.jsx

```
<div className="space-y-4">
  {comments.map((c) => (
    <div
      key={c.id}
      className="bg-[#2a2b4a] rounded-md p-3 flex justify-between items-start"
    >
      <div className="flex-1">
        <div className="flex items-center mb-1">
          <img
            src={c.profileIcon || "/icons/default.png"}
            className="w-4 h-4 mr-1"
            alt="icon"
          />
          <span className="font-bold text-sm text-blue-300 mr-2">
            {c.authorAlias}
          </span>
          <span className="text-xs text-gray-400">
            {new Date(c.createdAt).toLocaleDateString()}
          </span>
        </div>
        <p>{c.content}</p>
        {isCommentAuthor(c) && (
          <div className="flex gap-2 mt-1 text-xs text-gray-400">
            <button
              onClick={() => handleEditComment(c.id, c.content)}
              className="hover:text-yellow-300"
            >
              ✎ 수정
            </button>
            <button
              onClick={() => handleDeleteComment(c.id)}
              className="hover:text-red-300"
            >
              🗑 삭제
            </button>
          </div>
        )}
      </div>
    </div>
  )}
</div>
```

PostDetailPage.jsx

```
<div className="flex flex-col items-center text-xs space-y-1 ml-4">
  <button
    onClick={() => handleCommentReaction(c.id, "like")}
    className={
      commentReactions[c.id] === "like"
        ? "text-blue-400"
        : "text-blue-300 hover:text-blue-400"
    }
  >
    👍 {c.likes || 0}
  </button>
  <button
    onClick={() => handleCommentReaction(c.id,
"dislike")}
    className={
      commentReactions[c.id] === "dislike"
        ? "text-red-400"
        : "text-red-300 hover:text-red-400"
    }
  >
    👎 {c.dislikes || 0}
  </button>
</div>
</div>
))}
</div>
```

PostDetailPage.jsx

```
{isLoggedIn ? (  
  <form onSubmit={handleAddComment} className="mt-6">  
    <input  
      name="comment"  
      maxLength={100}  
      placeholder="100자 이내로 댓글 작성"  
      className="w-full bg-[#2a2b4a] px-3 py-2 rounded-md text-white placeholder-gray-400"  
    />  
    <br/><br/>  
    <button  
      type="submit"  
      className="mt-2 bg-pink-500 hover:bg-pink-600 px-4 py-2 rounded-md font-bold"  
    >  
      댓글 달기  
    </button>  
    <br/><br/>  
    <button  
      onClick={() => navigate(-1)}  
      className="mt-2 bg-gray-500 hover:bg-pink-600 px-4 py-2 rounded-md font-bold"  
    >  
      뒤로가기  
    </button>  
  </form>  
) : (  
  <p className="mt-6 text-gray-400">  
    댓글을 작성하려면{" "}   
    <a href="/login" className="underline text-blue-400">  
      로그인  
    </a>{" "}   
    이 필요합니다.  
  </p>  
  )}  
</div>  
</div>  
);  
}
```

SignupPage.jsx

```
import { useState, useRef, useEffect } from "react";
import LogoBlock from "../components/LogoBlock";

export default function SignupPage() {
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");
  const [errorMsg, setErrorMsg] = useState("");
  const [successMsg, setSuccessMsg] = useState("");
  const [selectedTheme, setSelectedTheme] = useState("밤손님");
  const usernameRef = useRef(null);

  const themes = ["밤손님", "마스터", "요정", "해결사", "바텐더"];

  useEffect(() => {
    usernameRef.current?.focus();
  }, []);

  const handleSignup = async (e) => {
    e.preventDefault();
    setErrorMsg("");
    setSuccessMsg("");

    if (password !== confirmPassword) {
      setErrorMsg("비밀번호가 일치하지 않습니다.");
      return;
    }
  }
}
```

SignupPage.jsx

```
try {
  const res = await fetch("http://localhost:8080/signup", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ username, password, selectedTheme }),
  });

  const msg = await res.text();

  if (res.ok) {
    setSuccessMsg("회원가입이 완료되었습니다!");
    setUsername("");
    setPassword("");
    setConfirmPassword("");
  } else {
    setErrorMsg(`회원가입 실패: ${msg}`);
  }
} catch {
  setErrorMsg("서버와 통신 중 문제가 발생했습니다.");
}
};
```

SignupPage.jsx

```
return (  
  <div className="min-h-screen bg-[#0b0c2a] text-white flex flex-col items-center justify-center px-4">  
    <LogoBlock />  
    <div className="bg-[#1a1b3a] p-6 rounded-xl w-full max-w-sm space-y-4 shadow-lg">  
      <h2 className="text-xl font-semibold text-center">회원가입</h2>  
      <form className="space-y-4" onSubmit={handleSignup}>  
        <div>  
          <label className="block mb-1 text-sm font-medium">아이디</label>  
          <input  
            ref={usernameRef}  
            type="text"  
            placeholder="영문 또는 숫자만 입력"  
            className="w-full p-2 rounded bg-white text-black"  
            required  
            value={username}  
            onChange={(e) => {  
              const onlyEnglish = e.target.value.replace(/^[^a-zA-Z0-9]/g, "");  
              setUsername(onlyEnglish);  
            }}  
          />  
        </div>  
  
        <div>  
          <label className="block mb-1 text-sm font-medium">비밀번호</label>  
          <input  
            type="password"  
            placeholder="비밀번호 (한글 입력 불가)"  
            className="w-full p-2 rounded bg-white text-black"  
            required  
            value={password}  
            onChange={(e) => {  
              const noKorean = e.target.value.replace(/[ㄱ-ㅎㅌ-ㅣ가-힣]/g, "");  
              setPassword(noKorean);  
            }}  
          />  
        </div>  
      </form>  
    </div>  
  </div>  
)
```

SignupPage.jsx

```
<div>
  <label className="block mb-1 text-sm font-medium">비밀번호 확인</label>
  <input
    type="password"
    placeholder="비밀번호 확인"
    className="w-full p-2 rounded bg-white text-black"
    required
    value={confirmPassword}
    onChange={(e) => {
      const noKorean = e.target.value.replace(/[ㄱ-ㅎㅏ-ㅣ가-힣]/g, "");
      setConfirmPassword(noKorean);
    }}
  />
</div>

<div>
  <label className="block mb-1 text-sm font-medium">테마 아이디 선택</label>
  <select
    className="w-full p-2 rounded bg-white text-black"
    value={selectedTheme}
    onChange={(e) => setSelectedTheme(e.target.value)}
  >
    {themes.map((theme, i) => (
      <option key={i} value={theme}>
        {theme}
      </option>
    ))}
  </select>
</div>
```

SignupPage.jsx

```
<button
  type="submit"
  className="w-full bg-green-500 hover:bg-green-400 text-white font-bold
py-2 rounded"
  >
    회원가입
  </button>
</form>

{errorMsg && <p className="text-red-400 text-center">{errorMsg}</p>}
{successMsg && <p className="text-green-400 text-center">{successMsg}</p>}

<button
  className="mt-4 w-full text-sm text-blue-300 hover:underline"
  onClick={() => (window.location.href = "/login")}
  >
    로그인 페이지로 돌아가기
  </button>
</div>
</div>
);
}
```

CorsConfig.java

```
package com.example.board.config;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
@Configuration
public class CorsConfig {
    private static final String[] ALLOWED_PATHS = {
        "/api/auth/**",
        "/api/comments/**",
        "/api/posts/**",
        "/api/test/**",
        "/api/users/**"
    };
    @Bean
    public WebMvcConfigurer corsConfigurer() {
        return new WebMvcConfigurer() {
            @Override
            public void addCorsMappings(CorsRegistry registry) {
                for (String path : ALLOWED_PATHS) {
                    registry.addMapping(path)
                        .allowedOrigins("http://localhost:3000")
                        .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
                        .allowCredentials(true);
                }
            }
        };
    }
}
```

WebConfig.java

```
package com.example.board.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("http://localhost:3000")
            .allowedMethods("GET", "POST", "PUT", "DELETE")
            .allowedHeaders("Content-Type", "Authorization")
            .allowCredentials(true);
    }
}
```

AuthController.java

```
package com.example.board.controller;

import com.example.board.domain.User;
import com.example.board.repository.UserRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import org.springframework.http.ResponseEntity;

import java.util.Map;
import java.util.Optional;
import java.util.List;

@RestController
@CrossOrigin(origins = "http://localhost:3000")
public class AuthController {

    @Autowired
    private UserRepository userRepository;

    @PostMapping("/signup")
    public ResponseEntity<String> signup(@RequestBody Map<String, String> payload) {
        String username = payload.get("username");
        String password = payload.get("password");
        String baseAlias = payload.get("selectedTheme");

        if (userRepository.findByUsername(username).isPresent()) {
            return ResponseEntity.badRequest().body("이미 존재하는 사용자입니다.");
        }

        List<User> existingUsers = userRepository.findByAliasStartingWith(baseAlias);
```

AuthController.java

```
int maxIndex = existingUsers.stream()
    .map(User::getAlias)
    .map(alias -> alias.replace(baseAlias, ""))
    .filter(suffix -> suffix.matches("\\d+"))
    .mapToInt(Integer::parseInt)
    .max()
    .orElse(0);

String alias = baseAlias + (maxIndex + 1);

User user = new User(username, password);
user.setAlias(alias);
userRepository.save(user);

return ResponseEntity.ok("회원가입 성공 (" + alias + ")");
}
@PostMapping("/login")
public ResponseEntity<?> login(@RequestBody User loginRequest) {
    Optional<User> foundUser = userRepository.findByUsername(loginRequest.getUsername());

    if (foundUser.isPresent() &&
        foundUser.get().getPassword().equals(loginRequest.getPassword())) {
        return ResponseEntity.ok(foundUser.get());
    }

    return ResponseEntity.status(401).body("로그인 실패");
}
}
```

CommentController.java

```
package com.example.board.controller;

import com.example.board.domain.Comment;
import com.example.board.repository.CommentRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Map;
import java.util.List;
import java.util.Optional;

@RestController
@CrossOrigin(origins = "http://localhost:3000", allowCredentials = "true")
public class CommentController {

    @Autowired
    private CommentRepository commentRepository;
```

CommentController.java

```
@PostMapping("/posts/{postId}/comments")
public Comment addComment(@PathVariable Long postId, @RequestBody Comment comment) {
    comment.setPostId(postId);
    return commentRepository.save(comment);
}

@GetMapping("/posts/{postId}/comments")
public List<Comment> getCommentsByPostId(@PathVariable Long postId) {
    return commentRepository.findByPostId(postId);
}

@PatchMapping("/comments/{id}")
public ResponseEntity<?> updateComment(@PathVariable Long id, @RequestBody Map<String, String> body) {
    Optional<Comment> optionalComment = commentRepository.findById(id);
    if (optionalComment.isEmpty()) return ResponseEntity.notFound().build();

    Comment comment = optionalComment.get();
    comment.setContent(body.get("content"));
    commentRepository.save(comment);

    return ResponseEntity.ok().build();
}

@DeleteMapping("/comments/{id}")
public ResponseEntity<?> deleteComment(@PathVariable Long id) {
    if (!commentRepository.existsById(id)) return ResponseEntity.notFound().build();
    commentRepository.deleteById(id);
    return ResponseEntity.ok().build();
}
}
```

PostController.java

```
package com.example.board.controller;

import com.example.board.domain.Post;
import com.example.board.repository.PostRepository;
import com.example.board.dto.PostDto;
import com.example.board.repository.LikeRepository;
import com.example.board.domain.Like;
import org.springframework.http.ResponseEntity;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDateTime;
import java.util.Optional;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

@RestController
@RequestMapping("/posts")
@CrossOrigin(origins = "http://localhost:3000", allowCredentials =
"true")
@Transactional
```

PostController.java

```
public class PostController {

    private final PostRepository postRepository;
    private final LikeRepository likeRepository;

    public PostController(PostRepository postRepository, LikeRepository likeRepository) {
        this.postRepository = postRepository;
        this.likeRepository = likeRepository;
    }

    @GetMapping("")
    public List<PostDto> getAllPosts() {
        return postRepository.findAll().stream()
            .map(post -> {
                PostDto dto = new PostDto();
                dto.setId(post.getId());
                dto.setCategory(post.getCategory());
                dto.setContent(post.getContent());
                dto.setAuthorAlias(post.getAuthorAlias());
                dto.setUserId(post.getUserId());
                dto.setTitle(post.getTitle());
                dto.setProfileIcon(post.getProfileIcon());
                dto.setLikes(post.getLikes());
                return dto;
            })
            .collect(Collectors.toList());
    }
}
```

PostController.java

```
@PostMapping("")
public ResponseEntity<Post> createPost(@RequestBody PostDto postDto) {
    Post post = new Post();
    post.setCategory(postDto.getCategory());
    post.setContent(postDto.getContent());
    post.setAuthorAlias(postDto.getAuthorAlias());
    post.setCreatedAt(LocalDateTime.now());
    post.setUserId(postDto.getUserId());
    post.setTitle(postDto.getTitle());
    post.setProfileIcon(postDto.getProfileIcon());
    post.setLikes(0);
    return ResponseEntity.ok(postRepository.save(post));
}

@PutMapping("/{postId}/like")
public ResponseEntity<String> toggleLike(@PathVariable Long postId, @RequestParam Long userId) {
    Optional<Post> postOptional = postRepository.findById(postId);
    if (postOptional.isEmpty()) return ResponseEntity.notFound().build();

    Post post = postOptional.get();
    boolean alreadyLiked = likeRepository.existsByUserIdAndPostId(userId, postId);

    if (alreadyLiked) {
        likeRepository.deleteByUserIdAndPostId(userId, postId);
        post.setLikes(Math.max(0, post.getLikes() - 1));
        postRepository.save(post);
        return ResponseEntity.ok("unliked");
    } else {
        Like like = new Like();
        like.setUserId(userId);
        like.setPostId(postId);
        likeRepository.save(like);

        post.setLikes(post.getLikes() + 1);
        postRepository.save(post);
        return ResponseEntity.ok("liked");
    }
}
```

UserController.java

```
package com.example.board.controller;

import com.example.board.domain.Post;
import com.example.board.domain.Comment;
import com.example.board.domain.Like;
import com.example.board.repository.PostRepository;
import com.example.board.repository.CommentRepository;
import com.example.board.repository.LikeRepository;

import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;

import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;

@RestController
@RequestMapping("/users")
@CrossOrigin(origins = "http://localhost:3000",
allowCredentials = "true")
```

UserController.java

```
public class UserController {

    private final PostRepository postRepository;
    private final CommentRepository commentRepository;
    private final LikeRepository likeRepository;

    @Autowired
    public UserController(PostRepository postRepository,
                          CommentRepository commentRepository,
                          LikeRepository likeRepository) {
        this.postRepository = postRepository;
        this.commentRepository = commentRepository;
        this.likeRepository = likeRepository;
    }

    @GetMapping("/{userId}/posts")
    public List<Post> getMyPosts(@PathVariable Long userId) {
        return postRepository.findById(userId);
    }

    @GetMapping("/{userId}/commented-posts")
    public List<Post> getCommentedPosts(@PathVariable Long userId) {
        List<Comment> comments = commentRepository.findById(userId);
        Set<Long> postIds = comments.stream()
            .map(Comment::getPostId)
            .collect(Collectors.toSet());
        return postRepository.findAllById(postIds);
    }
}
```

UserController.java

```
@GetMapping("/{userId}/liked-posts")
public List<Post> getLikedPosts(@PathVariable Long userId) {
    List<Like> likes = likeRepository.findByUserId(userId);
    Set<Long> postIds = likes.stream()
        .map(Like::getPostId)
        .collect(Collectors.toSet());
    return postRepository.findAllById(postIds);
}
```

Comment.java

```
package com.example.board.domain;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "comments")
public class Comment {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "post_id", nullable = false)
    private Long postId;

    @Column(name = "user_id")
    private Integer userId;

    @Column(name = "author_alias")
    private String authorAlias;

    @Column(name = "profile_icon")
    private String profileIcon;

    @Column(columnDefinition = "TEXT", nullable = false)
    private String content;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @Column(name = "author")
    private String author;
```

```
    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    @PrePersist
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
    }
```

Comment.java

```
public Long getId() {
    return id;
}

public Long getPostId() {
    return postId;
}

public void setPostId(Long postId) {
    this.postId = postId;
}

public Integer getUserId() {
    return userId;
}

public void setUserId(Integer userId) {
    this.userId = userId;
}

public String getAuthorAlias() {
    return authorAlias;
}

public void setAuthorAlias(String authorAlias) {
    this.authorAlias = authorAlias;
}

public String getProfileIcon() {
    return profileIcon;
}

public void setProfileIcon(String profileIcon) {
    this.profileIcon = profileIcon;
}

public String getContent() {
    return content;
}
```

```
public void setContent(String content) {
    this.content = content;
}

public LocalDateTime getCreatedAt() {
    return createdAt;
}

public void setCreatedAt(LocalDateTime createdAt) {
    this.createdAt = createdAt;
}
}
```

Like.java

```
package com.example.board.domain;

import jakarta.persistence.*;

@Entity
@Table(name = "user_likes", uniqueConstraints = {
    @UniqueConstraint(columnNames = {"user_id", "post_id"})
})
public class Like {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "user_id")
    private Long userId;

    @Column(name = "post_id")
    private Long postId;

    public Like() {}

    public Like(Long userId, Long postId) {
        this.userId = userId;
        this.postId = postId;
    }

    public Long getId() {
        return id;
    }

    public Long getUserId() {
        return userId;
    }

    public void setUserId(Long userId) {
        this.userId = userId;
    }

    public Long getPostId() {
        return postId;
    }

    public void setPostId(Long postId) {
        this.postId = postId;
    }
}
```

Post.java

```
package com.example.board.domain;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "posts")
public class Post {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "user_id")
    private Long userId;

    private String title;
    private String category;

    @Column(columnDefinition = "TEXT")
    private String content;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @Column(name = "likes")
    private Integer likes = 0;

    public int getLikes() {
        return likes;
    }

    public void setLikes(int likes) {
        this.likes = likes;
    }
}
```

Post.java

```
@PrePersist
protected void onCreate() {
    this.createdAt = LocalDateTime.now();
}

private String profileIcon;
private String authorAlias;

public Post() {
    this.createdAt = LocalDateTime.now();
}

public Long getId() {
    return id;
}

public Long getUserId() {
    return userId;
}

public void setUserId(Long userId) {
    this.userId = userId;
}

public String getTitle() {
    return title;
}

public void setTitle(String title) {
    this.title = title;
}

public String getCategory() {
    return category;
}

public void setCategory(String category) {
    this.category = category;
}

public String getContent() {
    return content;
}
```

```
public void setContent(String content) {
    this.content = content;
}

public LocalDateTime getCreatedAt() {
    return createdAt;
}

public void setCreatedAt(LocalDateTime createdAt) {
    this.createdAt = createdAt;
}

public String getProfileIcon() {
    return profileIcon;
}

public void setProfileIcon(String profileIcon) {
    this.profileIcon = profileIcon;
}

public String getAuthorAlias() {
    return authorAlias;
}

public void setAuthorAlias(String authorAlias) {
    this.authorAlias = authorAlias;
}
}
```

User.java

```
package com.example.board.domain;

import jakarta.persistence.*;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true)
    private String username;

    @Column(unique = true)
    private String alias;

    private String password;

    public User() {}

    public User(String username, String password) {
        this.username = username;
        this.password = password;
    }
}
```

```
public Long getId() {
    return id;
}

public String getUsername() {
    return username;
}

public void setUsername(String username) {
    this.username = username;
}

public String getAlias() {
    return alias;
}

public void setAlias(String alias) {
    this.alias = alias;
}

public String getPassword() {
    return password;
}

public void setPassword(String password) {
    this.password = password;
}
}
```

PostDto.java

```
package com.example.board.dto;

public class PostDto {
    private Long id;
    private String category;
    private String content;
    private String authorAlias;
    private Long userId;
    private String title;
    private String profileIcon;
    private int likes;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getProfileIcon() { return profileIcon; }
    public void setProfileIcon(String profileIcon) { this.profileIcon = profileIcon; }

    public PostDto() {
    }

    public Long getUserId() {
        return userId;
    }

    public void setUserId(Long userId) {
        this.userId = userId;
    }

    public String getCategory() {
        return category;
    }

    public void setCategory(String category) {
        this.category = category;
    }
}
```

```
public String getContent() {
    return content;
}

public void setContent(String content) {
    this.content = content;
}

public String getAuthorAlias() {
    return authorAlias;
}

public void setAuthorAlias(String authorAlias) {
    this.authorAlias = authorAlias;
}

public int getLikes() {
    return likes;
}

public void setLikes(int likes) {
    this.likes = likes;
}

public static PostDto from(com.example.board.domain.Post post) {
    PostDto dto = new PostDto();
    dto.setCategory(post.getCategory());
    dto.setContent(post.getContent());
    dto.setAuthorAlias(post.getAuthorAlias());
    dto.setUserId(post.getUserId());
    dto.setTitle(post.getTitle());
    dto.setProfileIcon(post.getProfileIcon());
    dto.setLikes(post.getLikes());
    return dto;
}
```

UserDto.java

```
package com.example.board.dto;

public class UserDto {
    private String username;
    private String password;

    public UserDto() {}

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}
```

UserDto.java

```
package com.example.board.model;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Post {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String category;
    private String content;
    private String author;
}
```

Comment, Like, Post, UserRepository.java

```
package com.example.board.repository;

import com.example.board.domain.Comment;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface CommentRepository extends
    JpaRepository<Comment, Long> {

    List<Comment> findByPostId(Long postId);
    List<Comment> findByUserId(Long userId);

}
```

```
package com.example.board.repository;

import com.example.board.domain.Post;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface PostRepository extends JpaRepository<Post,
    Long> {

    List<Post> findByUserId(Long userId);

}
```

```
package com.example.board.repository;

import com.example.board.domain.Like;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface LikeRepository extends JpaRepository<Like,
    Long> {

    boolean existsByUserIdAndPostId(Long userId, Long
    postId);

    void deleteByUserIdAndPostId(Long userId, Long postId);
    List<Like> findByUserId(Long userId);

}
```

```
package com.example.board.repository;

import com.example.board.domain.User;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;
import java.util.Optional;

public interface UserRepository extends JpaRepository<User,
    Long> {

    Optional<User> findByUsername(String username);
    boolean existsByAlias(String alias);

    List<User> findByAliasStartingWith(String aliasPrefix);

}
```

UserDto.java

```
package com.example.board.model;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Post {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String category;
    private String content;
    private String author;
}
```

출처

MVC 모델 사진 : <https://velog.io/@h220101/SpringBoot-%EC%8A%A4%ED%94%84%EB%A7%81-%EB%B6%80%ED%8A%B8-spring-MVC-%ED%8C%A8%ED%84%B4-%EB%8F%99%EC%9E%91>

Chat gpt